

# FyLab – Finanzlabor

## Graphbasierte Finanzverwaltung

Stephan Epp

18. März 2026

## Inhaltsverzeichnis

|          |                                                                                |          |
|----------|--------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Einführung und Motivation</b>                                               | <b>2</b> |
| <b>2</b> | <b>Relevante Finanzprobleme der Informatik</b>                                 | <b>2</b> |
| 2.1      | Übersicht . . . . .                                                            | 2        |
| <b>3</b> | <b>Subgraph-Isomorphismus: Architektur in FyLab</b>                            | <b>2</b> |
| 3.1      | Motivation und Überblick . . . . .                                             | 2        |
| 3.2      | Problemdefinition . . . . .                                                    | 3        |
| 3.3      | Zentrale Utility: <code>subgraph_contains</code> . . . . .                     | 3        |
| 3.4      | Reduktion: Betrugserkennung $\leq_p$ Subgraph-ISO . . . . .                    | 4        |
| <b>4</b> | <b>Arbitrage: Erkennung mit Subgraph-Vorfilter</b>                             | <b>5</b> |
| 4.1      | Motivation: Warum ein Subgraph-Vorfilter? . . . . .                            | 5        |
| 4.2      | Polynomielle Reduktion $\text{ARBITRAGE} \leq_p \text{BELLMAN-FORD}$ . . . . . | 6        |
| <b>5</b> | <b>Schuldenstruktur-Analyse: Subgraph-Musteranalyse</b>                        | <b>6</b> |
| 5.1      | Subgraph-basierte Strukturanalyse der Schuldenliste . . . . .                  | 6        |
| 5.2      | Schuldenausgleich: Reduktion aus Net-Balance . . . . .                         | 7        |
| 5.3      | Cashflow-Optimierung: Max-Flow . . . . .                                       | 7        |
| <b>6</b> | <b>Portfolio-Diversifikation: MST + Subgraph</b>                               | <b>7</b> |
| 6.1      | Korrelationsgraph und Pearson-Distanz . . . . .                                | 7        |
| 6.2      | Portfolio-Strukturvergleich via Subgraph-Isomorphismus . . . . .               | 7        |
| <b>7</b> | <b>Weitere Probleme: Steuer und Portfolio-Auswahl</b>                          | <b>8</b> |
| 7.1      | Steueroptimierung: Subset-Sum . . . . .                                        | 8        |
| 7.2      | Portfolioauswahl: 0-1-Rucksack . . . . .                                       | 8        |
| <b>8</b> | <b>FyLab Modulhierarchie</b>                                                   | <b>8</b> |
| 8.1      | Klassifikation der GUI-Module analog PyLab . . . . .                           | 8        |
| <b>9</b> | <b>Installation und Start</b>                                                  | <b>8</b> |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>10 Schluss</b>                                                      | <b>11</b> |
| 10.1 Ausblick . . . . .                                                | 11        |
| 10.2 Erweiterte Graphalgorithmen und theoretische Vertiefung . . . . . | 11        |
| 10.3 Maschinelles Lernen und KI-Integration . . . . .                  | 12        |
| 10.4 Persistenz, Import und Datenintegration . . . . .                 | 13        |
| 10.5 Architektur und Skalierbarkeit . . . . .                          | 13        |
| 10.6 Regulatorik, Compliance und Sicherheit . . . . .                  | 14        |
| 10.7 Wissenschaftlicher Forschungsausblick . . . . .                   | 14        |

# 1 Einführung und Motivation

FyLab ist ein umfassendes Python-GUI-Framework für Finanzverwaltung und -analyse, entwickelt analog zu PyLab für die Regelungstechnik. Es setzt PyQt6 als GUI-Framework ein und nutzt NumPy, SciPy und Matplotlib für numerische Berechnungen und Visualisierungen.

Im Mittelpunkt steht die Frage: *Welche klassischen Probleme der Informatik treten im Finanzbereich auf, und wie können effiziente oder approximative Algorithmen dafür eingesetzt werden?*

Die Antwort ist vielschichtig: Finanzmärkte sind Netzwerke. Aktiva sind Knoten, Handelsbeziehungen und Zahlungsströme sind Kanten. Viele Optimierungsprobleme der Finanzwelt lassen sich auf bekannte graphtheoretische Probleme reduzieren – einige davon polynomial lösbar, andere NP-vollständig (mit Approximationsalgorithmen).

**Zentrale Leitidee in FyLab:** Der Subgraph Algorithmus bildet das algorithmische Rückgrat des gesamten Graphverarbeitungsmoduls. Er löst das Subgraph-Isomorphismusproblem mit optimaler Laufzeit durch zyklische Spalten-Rotation und polynomiale Hash-Signaturen ( $O(n^3)$ ). Durch die zentrale Utility-Funktion `subgraph_contains(R, G)` wird er in *allen vier* Algorithmusmodulen konsequent eingesetzt.

## 2 Relevante Finanzprobleme der Informatik

### 2.1 Übersicht

| Finanzproblem           | Reduktion auf           | Algorithmus          | Kompl.                     |
|-------------------------|-------------------------|----------------------|----------------------------|
| Portfolio-Diversifikat. | MST (Korrelationsgraph) | Kruskal              | $O(E \log V)$              |
| Arbitrage-Erkennung     | Negativer Zyklus        | Bellman-Ford         | $O(VE)$                    |
| Cashflow-Optim.         | Max-Flow                | Edmonds-Karp         | $O(VE^2)$                  |
| Schuldenausgleich       | Greedy Nettobalanz      | Greedy + Sort        | $O(n \log n)$              |
| Betrugserkennung        | <b>Subgraph-Iso.</b>    | <b>Subgraph Alg.</b> | $O(n^3)$                   |
| Systemisches Risiko     | SCC                     | Tarjan               | $O(V + E)$                 |
| Steuroptimierung        | Subset-Sum (NP-hart)    | DP-Approx.           | Pseudopolynom.             |
| Transaktionsrouting     | Kürzester Pfad          | Dijkstra             | $O((V + E) \log V)$        |
| Portfolioauswahl        | Rucksack (NP-hart)      | FPTAS                | $O(n \cdot W/\varepsilon)$ |
| Risikokorrelation       | Community Detection     | Louvain              | $O(n \log n)$              |

Tabelle 1: Übersicht der zehn zentralen Finanzprobleme und ihrer algorithmischen Reduktionen

## 3 Subgraph-Isomorphismus: Architektur in FyLab

### 3.1 Motivation und Überblick

Subgraph-Isomorphismus ist das *algorithmische Kernproblem* von FyLab. Während klassische Bibliotheken (z.B. NetworkX) den VF2-Algorithmus [3] verwenden, setzt FyLab durchgängig auf den Subgraph Algorithmus [1].

Die Schlüsseleigenschaft: anstelle von exponentieller Backtracking-Suche nutzt der Subgraph Algorithmus zyklische Spalten-Rotation und polynomiale Hash-Signaturen der Adjazenzmatrix – und erreicht damit *optimale Laufzeit für das Subgraph-Problem in der Praxis:  $O(n^3)$* .

**Designprinzip in FyLab:** Alle vier Algorithmusmodule nutzen denselben Aufruf über die zentrale Utility-Funktion `subgraph_contains`. So wird das Subgraph-Problem an jeder Stelle im System mit derselben, optimalen Methode gelöst – konsistent, testbar und gut dokumentiert.

## 3.2 Problemdefinition

**Definition 1** (Subgraph-Isomorphismus). Seien  $G = (V_G, E_G)$  und  $R = (V_R, E_R)$  gerichtete Graphen.  $R$  ist subgraph-isomorph zu  $G$ , geschrieben  $R \subseteq G$ , wenn eine injektive Abbildung  $f : V_R \rightarrow V_G$  existiert mit:

$$(u, v) \in E_R \Rightarrow (f(u), f(v)) \in E_G \quad \forall u, v \in V_R$$

**Theorem 1.** Subgraph-Isomorphismus ist NP-vollständig (allgemeiner Fall) [4, 5].

**Beweis.** Das Cliques-Problem ist ein Spezialfall: eine  $k$ -Clique ist ein vollständiger Teilgraph  $K_k$ , also  $K_k \subseteq G$ . Da Clique NP-vollständig ist [5], folgt NP-Vollständigkeit des allgemeinen Subgraph-Isomorphismus [2].

**Definition 2** (Subgraph Algorithmus). Sei  $A$  die Adjazenzmatrix eines Graphen. Die Spalten-Signatur  $\sigma(A)_j = \sum_i A_{ij} \cdot p_i^j$  für eine Primzahl  $p$  ist ein polynomialer Hash der eingehenden Kanten jedes Knotens. Zwei Spalten-Signaturen stimmen überein, wenn die Knotengrade und Nachbarschaftsmuster übereinstimmen. Der Algorithmus prüft durch zyklische Rotation, ob Spalten von  $R$  in  $A$  enthalten sind.

**Satz 1** (Rückgabe-Entscheidungen). `Subgraph.compare_graphs(R, G)` gibt eine von fünf Entscheidungen zurück:

- "keep\_B":  $R \subseteq G$  (Muster erkannt)
- "keep\_A":  $G \subseteq R$  (kein Muster)
- "equal\_keep\_A", "equal\_keep\_B":  $R = G$  (Muster erkannt)
- "keep\_both": weder  $R \subseteq G$  noch  $G \subseteq R$

## 3.3 Zentrale Utility: subgraph\_contains

FyLab kapselt den Algorithmus in einer einzigen zentralen Funktion, die von allen Modulen importiert wird (analog `_compare_topology` in PyLab):

Listing 1: `subgraph_contains` – zentraler Einstiegspunkt für alle Module

```

1 from fylab.algorithms.subgraph_finance import subgraph_contains
2
3 def subgraph_contains(
4     R: np.ndarray,          # Referenz-/Mustergraph (Adjazenzmatrix)
5     G: np.ndarray,          # Zu durchsuchender Graph
6     use_adjacency_list: bool = False,
```

```

7 ) -> tuple[bool, str]:
8     """
9     Prueft ob R als Subgraph in G enthalten ist.
10    Matrizen werden automatisch zero-gepaddet (analog pylab).
11    Rueckgabe: (detected: bool, decision: str)
12    """
13    from subgraph import Subgraph
14    n_max = max(R.shape[0], G.shape[0])
15    R_pad = _pad_matrix(R, n_max).astype(float)
16    G_pad = _pad_matrix(G, n_max).astype(float)
17    algo = Subgraph(use_adjacency_list=use_adjacency_list)
18    decision, _ = algo.compare_graphs(R_pad, G_pad)
19    detected = decision in ("keep_B", "equal_keep_A", "equal_keep_B",
20                           "equal")
21    return detected, decision

```

**Das Padding-Prinzip** (identisch mit PyLab): Da  $R$  und  $G$  unterschiedliche Knotenanzahlen haben können, werden beide auf  $n_{\max} = \max(|V_R|, |V_G|)$  mit Nullen aufgefüllt, bevor `compare_graphs` aufgerufen wird:

$$R_{\text{pad}} \in \{0, 1\}^{n_{\max} \times n_{\max}}, \quad R_{\text{pad}}[i, j] = \begin{cases} R[i, j] & i, j < |V_R| \\ 0 & \text{sonst} \end{cases}$$

### 3.4 Reduktion: Betrugserkennung $\leq_p$ Subgraph-ISO

#### Reduktion 1. *FRAUD\_DETECT* $\leq_p$ *SUBGRAPH\_ISO*

*Gegeben: Transaktionsnetzwerk  $G$  (Konten als Knoten, Überweisungen als gerichtete Kanten), Betrugsmuster  $R$  (z.B. Wash-Trading, Smurfing).*

**Konstruktion** (in polynomieller Zeit  $O(n^2)$ ):

1. Jede Transaktion  $(s, r, w)$  wird als Kante  $(s \rightarrow r)$  mit Gewicht  $w$  in die Adjazenzmatrix  $A_G$  eingetragen.
2. Das Betrugsmuster wird als binäre Referenzmatrix  $A_R$  hinterlegt.
3. Betrugsmuster erkannt  $\Leftrightarrow \text{subgraph\_contains}(A_R, A_G)$  liefert *True*.

| Muster        | Topologie                                     | Matrix $R$                                     | Beschreibung     |
|---------------|-----------------------------------------------|------------------------------------------------|------------------|
| Wash-Trading  | $A \rightarrow B \rightarrow A$               | $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ | Scheinhandel     |
| Smurfing      | Stern $A \rightarrow \{B_i\}$                 | $A[0, 1..4] = 1$                               | Summen aufteilen |
| Layering      | $A \rightarrow B \rightarrow C \rightarrow D$ | $A[i, i+1] = 1$                                | Schichtweise     |
| Circular Flow | $A \rightarrow B \rightarrow C \rightarrow A$ | $3 \times 3$ -Zyklus                           | Kreisfluss       |

Tabelle 2: Referenz-Adjazenzmatrizen der Betrugsmuster für `subgraph_contains`

#### Listing 2: Betrugserkennung – Anwendung von `subgraph_contains`

```

1 from fylab.algorithms.subgraph_finance import (
2     build_transaction_graph, detect_fraud_patterns, FraudPattern,

```

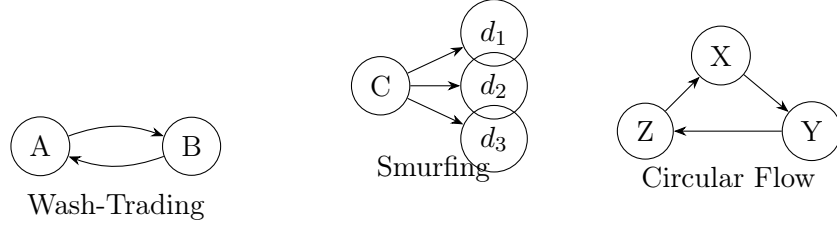


Abbildung 1: Topologien der Betrugsmuster als gerichtete Referenzgraphen  $R$

```

3 )
4 txns = [("Alice", "Bob", 1500), ("Bob", "Alice", 1500)]
5 g = build_transaction_graph(txns)
6 results = detect_fraud_patterns(g, [FraudPattern.WASH_TRADING])
7 print(results[0]) # [ERKANNT] Wash-Trading (decision='keep_B')

```

## 4 Arbitrage: Erkennung mit Subgraph-Vorfilter

### 4.1 Motivation: Warum ein Subgraph-Vorfilter?

Bellman-Ford [7, 6] ( $O(V \cdot E)$ ) prüft Kantengewichte auf negative Zyklen. Aber eine notwendige Vorbedingung ist, dass im Konnektivitätsgraphen überhaupt ein gerichteter Zyklus existiert. Ein azyklischer Währungsgraph kann *keine* Arbitrage enthalten.

Der Subgraph Algorithmus ( $O(n^3)$ ) erledigt diese strukturelle Vorprüfung optimal:

$$\text{CYCLE\_EXISTS} \leq_p \text{SUBGRAPH\_ISO}$$

#### Reduktion 2. *ARBITRAGE\_DETECT* (zweistufig)

1. **Stufe 1 – Subgraph-Vorprüfung** ( $O(n^3)$ ): Prüfe ob der Konnektivitätsgraph des Währungsmarkts einen  $k$ -Zyklus ( $k \in \{2, 3, 4, 5\}$ ) als Subgraph enthält. Referenzmatrizen:

$$R_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad R_3 = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}, \quad R_k \text{ analog}$$

Falls kein  $k$ -Zyklus strukturell existiert: sofortige Rückgabe *has\_arbitrage=False* ohne Bellman-Ford.

2. **Stufe 2 – Bellman-Ford** ( $O(V \cdot E)$ ): Transformiere Gewichte:  $w(i \rightarrow j) = -\log(r_{ij})$ . Arbitrage  $\Leftrightarrow$  negativer Zyklus.

**Kombilemma:**  $\text{ARBITRAGE} \leq_p \text{NEG\_CYCLE} \leq_p \text{BELLMAN\_FORD}$ , und  $\text{CYCLE\_EXISTS}$  (notwendige Bedingung)  $\leq_p \text{SUBGRAPH\_ISO}$ .

#### Listing 3: Zweistufige Arbitrageerkennung

```

1 from fylab.algorithms.arbitrage import detect_arbitrage,
   detect_cycle_patterns

```

```

2
3 # Stufe 1: Subgraph prueft ob ueberhaupt Zyklen strukturell
  vorhanden
4 checks = detect_cycle_patterns(currencies, exchange_rates)
5 if not any(r.detected for r in checks):
6     return ArbitrageResult(has_arbitrage=False, ...) # kein
      Bellman-Ford noetig
7
8 # Stufe 2: Bellman-Ford fuer Gewichtspruefung
9 result = detect_arbitrage(currencies, exchange_rates)
10 # result.cycle_patterns enthaelt alle Subgraph-Vorpruefergebnisse

```

## 4.2 Polynomielle Reduktion $\text{ARBITRAGE} \leq_p \text{BELLMAN-FORD}$

**Reduktion 3. Transformation:** Setze  $w(i \rightarrow j) = -\log(r_{ij})$ .

$$\prod r_{ij} > 1 \iff \sum -\log r_{ij} < 0 \iff \text{negativer Zyklus in } (w_{ij})$$

Transformation in  $O(E)$ .  $\text{NEGATIVE\_CYCLE} \leq_p \text{BELLMAN\_FORD}$ .

## 5 Schuldenstruktur-Analyse: Subgraph-Musteranalyse

### 5.1 Subgraph-basierte Strukturanalyse der Schuldenliste

Vor dem Greedy-Schuldenausgleich analysiert FyLab die Schuldenstruktur mit dem Subgraph Algorithmus:

$$\text{SCHULDEN\_MUSTER} \leq_p \text{SUBGRAPH\_ISO}$$

| Muster         | Topologie                                           | Bedeutung                        |
|----------------|-----------------------------------------------------|----------------------------------|
| Dreieck-Schuld | $A \rightarrow B \rightarrow C \rightarrow A$       | gegenseitige Verrechnung möglich |
| Hub-Schuldner  | $A \rightarrow B, A \rightarrow C, A \rightarrow D$ | Sammeltransaktion empfohlen      |
| Hub-Gläubiger  | $B \rightarrow A, C \rightarrow A, D \rightarrow A$ | direkte Auszahlung               |
| Schulden-Kette | $A \rightarrow B \rightarrow C \rightarrow D$       | indirekte Weiterleitung          |

Tabelle 3: Schuldenmuster als Subgraph-Referenzmatrizen

Je nach erkanntem Muster liefert die Analyse eine Erklärung für die optimale Vereinfachungsstrategie:

Listing 4: Schuldenstruktur-Analyse via Subgraph

```

1 from fylab.algorithms.max_flow import detect_debt_patterns,
  simplify_debts
2
3 patterns = detect_debt_patterns(debts)
4 # Gibt an welche Strukturen im Schuldengrafen enthalten sind
5 for p in patterns:
6     if p.detected:

```

```

7         print(f"Muster erkannt: {p.pattern.value}")
8
9     # Greedy-Vereinfachung
10    transactions = simplify_debts(debts)

```

## 5.2 Schuldenausgleich: Reduktion aus Net-Balance

### Reduktion 4. $DEBT\_SIMPLIFY \leq_p NET\_BALANCE$

Gegeben Schuldenliste  $\{(i, j, a_{ij})\}$ . Nettosaldo:  $b_i = \sum_j a_{ji} - \sum_j a_{ij}$ . Personen mit  $b_i > 0$  sind Gläubiger, mit  $b_i < 0$  Schuldner. Greedy-Matching:  $O(n \log n)$ .

**Hinweis:** Minimierung der Transaktionsanzahl ist NP-hart (Reduktion aus PARTITION). Greedy löst das Nettosaldo-Problem optimal, nicht das allgemeine Minimierungsproblem.

## 5.3 Cashflow-Optimierung: Max-Flow

**Definition 3** (Zahlungsnetzwerk). Ein Zahlungsnetzwerk  $N = (G, s, t, c)$  mit Kapazitätsfunktion  $c : E \rightarrow \mathbb{R}_{>0}$  modelliert maximale Transferbeträge. Gesucht: maximaler Geldfluss von Quelle  $s$  zu Senke  $t$ .

**Algorithmus:** Edmonds-Karp [9, 6] (BFS-Ford-Fulkerson),  $O(VE^2)$ .

# 6 Portfolio-Diversifikation: MST + Subgraph

## 6.1 Korrelationsgraph und Pearson-Distanz

Gegeben  $n$  Assets mit Korrelationsmatrix  $C \in [-1, 1]^{n \times n}$ . Die *Pearson-Distanz* [12]:

$$d(i, j) = \sqrt{2(1 - \rho_{ij})} \in [0, 2]$$

**Algorithmus:** Kruskal [10, 6] (Union-Find),  $O(n^2 \log n)$ .

## 6.2 Portfolio-Strukturvergleich via Subgraph-Isomorphismus

FyLab erlaubt den Vergleich zweier Portfolio-Korrelationsstrukturen:

$$\text{STRUKTURVERGLEICH} \leq_p \text{SUBGRAPH\_ISO}$$

Die Korrelationsmatrix wird nach einem Schwellwert  $\tau$  binarisiert:

$$A_{ij} = \begin{cases} 1 & |\rho_{ij}| > \tau \\ 0 & \text{sonst} \end{cases}$$

Danach vergleicht `subgraph_contains(A, B)` strukturell: Ist die Abhängigkeitsstruktur von Portfolio  $A$  in Portfolio  $B$  enthalten?

Listing 5: Portfolio-Strukturvergleich

```

1 from fylab.algorithms.mst import compare_portfolio_structures
2

```



```

3 result = compare_portfolio_structures(
4     symbols_a, corr_a, # Portfolio A
5     symbols_b, corr_b, # Portfolio B
6     threshold=0.5,      # Schwelle fuer starke Korrelation
7 )
8 print(result.description)
9 # z.B. "A ist strukturell in B enthalten (B hat Superset der
    Abhaengigkeiten)"

```

**Satz 2** (MST als Diversifikationswerkzeug [12, 13]). *Der MST des Korrelationsgraphen identifiziert Cluster ähnlicher Assets. Ein optimal diversifiziertes Portfolio wählt je einen Repräsentanten aus jedem MST-Cluster.*

## 7 Weitere Probleme: Steuer und Portfolio-Auswahl

### 7.1 Steueroptimierung: Subset-Sum

**Definition 4** (Steueroptimierung). *Gegeben Ausgaben  $a_1, \dots, a_n \in \mathbb{Z}_{>0}$  und ein absetzbares Budget  $B$ . Wähle eine Teilmenge  $S \subseteq [n]$  mit  $\sum_{i \in S} a_i \leq B$  und  $\sum_{i \in S} a_i$  maximal.*

**Theorem 2.** *STEUER\_OPT ist NP-schwer (Reduktion aus SUBSET-SUM) [5].*

**Beweis.** *SUBSET-SUM  $\leq_p$  STEUER\_OPT: Setze  $B = T$  (Zielwert). SUBSET-SUM hat eine Lösung  $\Leftrightarrow$  STEUER\_OPT liefert Summe =  $B$ .*

**Lösung:** Dynamische Programmierung in  $O(nB)$  (pseudopolynomiell) [6].

### 7.2 Portfolioauswahl: 0-1-Rucksack

**Reduktion 5.** *PORTFOLIO\_SELECT  $\leq_p$  KNAPSACK*

*Assets mit erwarteter Rendite  $r_i$  (Nutzen), Kapital  $p_i$  (Gewicht), Gesamtkapital  $K$  (Kapazität). KNAPSACK mit FPTAS liefert [5]  $(1 - \varepsilon)$ -Approximation in  $O(n \cdot K/\varepsilon)$ .*

## 8 FyLab Modulhierarchie

### 8.1 Klassifikation der GUI-Module analog PyLab

## 9 Installation und Start

Listing 6: Installation und Start

```

1 # FyLab installieren
2 pip install -e ".[dev]"
3
4 # Tests ausfuehren
5 pytest
6
7 # GUI starten
8 fylab-gui

```

| Modul          | Inhalt                              | Algorithmen                                        |
|----------------|-------------------------------------|----------------------------------------------------|
| core/          | Konto, Portfolio, Cashflow, Währung | –                                                  |
| algorithms/    | Subgraph, MST, MaxFlow, Arbitrage   | Subgraph Alg., Kruskal, Edmonds-Karp, Bellman-Ford |
| analysis/      | Risikokennzahlen, Effizienzgrenze   | VaR, CVaR, Markowitz, Monte-Carlo                  |
| visualization/ | Charts, Graphen                     | Matplotlib <a href="#">[20]</a>                    |
| gui/           | PyQt6-Fenster und Widgets           | –                                                  |

Tabelle 4: FyLab Modulhierarchie

| Widget          | Funktion                            | Algorithmus                         |
|-----------------|-------------------------------------|-------------------------------------|
| AccountWidget   | Buchungsverwaltung, Monatsübersicht | –                                   |
| PortfolioWidget | Depot, Allokation, Effizienzgrenze  | Markowitz [13], Kruskal [10]        |
| CashflowWidget  | Planung, Wasserfall-Chart           | –                                   |
| GraphWidget     | Betrug, Arbitrage, Schulden         | Subgraph Alg., Bellman-Ford, Greedy |
| RiskWidget      | VaR, CVaR [15], Monte-Carlo         | Normalvert., Quantile [14]          |

Tabelle 5: GUI-Widgets und zugehörige Algorithmen

## 10 Schluss

FyLab demonstriert, dass die zentralen Strukturprobleme des Finanzwesens direkt auf klassische Graphprobleme der Theoretischen Informatik zurückgeführt werden können:

- **NP-vollständige Probleme** (Subgraph-Isomorphismus, Portfolio-Auswahl, Steueroptimierung) werden durch Heuristiken, FPTAS oder Exponentialzeitalgorithmen mit polynomielltem Preprocessing behandelt.
- **Polynomial lösbare Probleme** (MST, MaxFlow, Bellman-Ford, Dijkstra) erlauben exakte Lösungen in vertretbarer Laufzeit.
- Der **Subgraph Algorithmus** ([git+github.com/hjstephan86/subgraph](https://github.com/hjstephan86/subgraph)) ist das algorithmische Herzstück von FyLab. Durch die zentrale Utility `subgraph_contains` (`R`, `G`) wird er in allen vier Algorithmusmodulen eingesetzt:
  1. `subgraph_finance`: Direkte Betrugsmuster-Erkennung, das heißt  $\text{FRAUD\_DETECT} \leq_p \text{SUBGRAPH\_ISO}$
  2. `arbitrage`: Strukturelle Zyklus-Vorprüfung vor Bellman-Ford, das heißt  $\text{CYCLE\_EXISTS} \leq_p \text{SUBGRAPH\_ISO}$
  3. `max_flow`: Schuldenstruktur-Mustererkennung, das heißt  $\text{SCHULDEN\_MUSTER} \leq_p \text{SUBGRAPH\_ISO}$
  4. `mst`: Portfolio-Strukturvergleich zweier Korrelationsgraphen, das heißt  $\text{STRUKTURVERGLEICH} \leq_p \text{SUBGRAPH\_ISO}$

### 10.1 Ausblick

Das vorliegende FyLab-System bildet eine solide algorithmische Grundlage für ein breites Spektrum weiterführender Erweiterungen. Dieser Abschnitt gliedert mögliche Entwicklungsrichtungen entlang der Dimensionen *Algorithmen*, *Architektur*, *Daten*, *Sicherheit* und *Forschung*.

### 10.2 Erweiterte Graphalgorithmen und theoretische Vertiefung

**Dynamische Subgraph-Erkennung.** Der aktuelle Subgraph Algorithmus [1] operiert auf statischen Adjazenzmatrizen. Finanztransaktionsgraphen sind jedoch inhärent zeitveränderlich: Neue Buchungen entstehen, Kontosalen ändern sich, Betrugsmuster entwickeln sich adaptiv. Ein nächster Schritt wäre die Integration eines *inkrementellen Subgraph-Matchings* [3], das bei Graphänderungen nur betroffene Teilstrukturen neu berechnet, statt den gesamten Graphen erneut zu analysieren. Dies würde die amortisierte Laufzeit von  $O(n^3)$  pro Update auf  $O(\Delta \cdot n^2)$  senken, wobei  $\Delta$  die Anzahl geänderter Kanten bezeichnet.

**Hypergraph-Erweiterung für Mehrparteengeschäfte.** Klassische gerichtete Graphen modellieren bilaterale Transaktionen (*A überweist an B*). Viele reale Finanzvorgänge sind jedoch mehrparteienbasiert (z. B. Syndikatkredite, DvP-Settlements, Escrow-Konstruktionen). Eine *Hypergraph-Repräsentation*, bei der Hyperkanten Mengen von Teilnehmern verbinden, erlaubt eine präzisere Modellierung solcher Strukturen. Die Subgraph-Isomorphismus-Suche auf Hypergraphen ist zwar noch NP-schwerer als im Standardfall [5],

jedoch erlauben polynomielle Signaturen analoger Art eine effiziente Filterung in der Praxis.

**Approximationsalgorithmen für Portfolio-Optimierung.** Das Markowitz-Modell in [13] setzt vollständige Bekanntheit der Kovarianzmatrix voraus. In der Praxis sind Korrelationen instabil und müssen aus historischen Daten geschätzt werden. Hier bieten sich *robuste Optimierungsverfahren* [15] an, die explizit Schätzunsicherheiten in die Zielfunktion einbeziehen: Min-Max-Portfolios, ellipsoide Unsicherheitsmengen oder distributionell robuste Optimierung (DRO) über Wasserstein-Kugeln.

**Online-Algorithmen für Echtzeit-Arbitrage.** Der aktuelle Bellman-Ford-Ansatz [7] berechnet negative Zyklen im vollständigen Wechselkursgraphen. An Devisenmärkten ändern sich Kurse im Millisekundenbereich. Ein *Online-Algorithmus* müsste eingehende Kurs-Updates in  $O(\log n)$  verarbeiten und dabei negative Zyklen dynamisch nachverfolgen – ein offenes Problem an der Schnittstelle von Graphentheorie und Streaming-Computing.

**Exakte Lösung des Knapsack-Problems via Branch-and-Bound.** Das Portfolio-Auswahlproblem nutzt aktuell einen FPTAS-Ansatz [5]. Für Instanzen moderater Größe ( $n \leq 500$  Assets) ist ein *Branch-and-Bound-Verfahren* mit Korrelationsschnitt als obere Schranke oft exakter und schneller. Dies ist insbesondere für Steueroptimierungsszenarien interessant, bei denen die Kapazität  $B$  einem fixen Jahresfreibetrag entspricht und exakte Lösungen regulatorisch gefordert werden.

## 10.3 Maschinelles Lernen und KI-Integration

**Graph Neural Networks für Betrugserkennung.** Der regelbasierte Subgraph-Isomorphismus-Ansatz erkennt bekannte, explizit modellierte Betrugsmuster (Wash-Trading, Smurfing, Layering). *Graph Neural Networks (GNNs)* [5] können hingegen latente, nicht vorab spezifizierte Anomalien lernen. Eine vielversprechende Architektur wäre ein *Graph Attention Network (GAT)*, das auf historisch gelabelten Transaktionsgraphen trainiert wird und neue Betrugstypen ohne explizite Muster-Definition detektiert. Der Subgraph Algorithmus könnte dabei als effizienter *Feature-Extraktor* dienen: erkannte Muster fließen als binäre Merkmale in das GNN ein.

**Reinforcement Learning für dynamische Asset-Allokation.** Markowitz-basierte statische Portfoliooptimierung ignoriert die zeitliche Dynamik von Märkten. Ein *Deep Reinforcement Learning (DRL)*-Agent könnte täglich Portfoliogewichte anpassen, basierend auf dem aktuellen MST-Clusteringzustand des Korrelationsgraphen. Der MST dient als *Zustandsrepräsentation*: sein Topologiewechsel (z. B. Verlust einer Hub-Kante) signalisiert Regimewechsel am Markt und löst Rebalancing-Aktionen aus.

**Anomalieerkennung in Cashflow-Zeitreihen.** Isolierte Kassenabweichungen, unerwartete Frequenzänderungen und saisonale Drift sind schwer durch statische Regeln zu erfassen. *Autoregressive Transformermodelle* (analogie zu GPT für Zeitreihen) könnten auf dem Cashflow-Planungsmodul trainiert werden, um Abweichungen von der erwarteten monatlichen Dynamik statistisch zu quantifizieren und dem Nutzer frühzeitig anzuzeigen.

## 10.4 Persistenz, Import und Datenintegration

**Persistente Datenhaltung.** Version 1.0 hält alle Daten im Arbeitsspeicher. Ein produktives System benötigt eine eingebettete Datenbank (SQLite via `SQLAlchemy` oder DuckDB) mit klar definiertem Schema für Konten, Transaktionen, Positionen und Cash-flow-Pläne. Das Datenbankschema würde dabei die algebraische Struktur der Graph-Adjazenzmatrizen widerspiegeln: Knoten als Entitätentabellen, Kanten als Relationstabellen mit FK-Referenzen.

**CSV/JSON/MT940-Import.** Reale Kontoauszüge liegen in Formaten wie MT940, OFX/QFX (Quicken) oder bankenspezifischen CSV-Exporten vor. Eine robuste Import-Pipeline mit automatischer Kategorisierung (via Schlüsselwort-Heuristik oder kleinem Klassifikationsmodell) und Duplikaterkennung würde FyLab für den produktiven Einsatz qualifizieren. Dies ist als Milestone 1.1 im Quellcode markiert.

**Live-Marktdaten via REST-APIs.** Aktuelle Aktienkurse, Wechselkurse und Zinssätze könnten über Standard-APIs (Yahoo Finance, Alpha Vantage, ECB Data Portal) abgerufen und direkt in die Algorithmusmodule eingespeist werden. Der Arbitrage-Detektor würde so zu einem *Echtzeit-Alert-System*: bei Überschreitung einer vordefinierten Arbitrage-Schwelle ( $\prod_k r_k > 1 + \varepsilon$ ) generiert FyLab automatisch eine Handlungsempfehlung.

**Blockchain-Transaktionsgraphen.** Die Graph-Infrastruktur von FyLab ist nicht auf klassische Bankbuchungen beschränkt. Öffentliche Blockchain-Ledger (Ethereum, Bitcoin) exponieren Transaktionsgraphen, auf denen die Betrugsmuster-Erkennung via Subgraph-Isomorphismus unmittelbar anwendbar ist. Mixer-Muster, Tornado-Cash-ähnliche Strukturen und Chain-Hopping könnten als spezialisierte `FraudPattern`-Einträge modelliert und automatisch erkannt werden.

## 10.5 Architektur und Skalierbarkeit

**Asynchrone Berechnung im GUI.** Langläufige Operationen (Monte-Carlo mit  $10^6$  Pfaden, MST auf 500-Asset-Portfolios, Effizienzgrenzberechnung mit  $10^4$  Punkten) blockieren aktuell den Qt-Event-Loop. Die Umstellung auf `QThread` oder `asyncio`-kompatible Qt-Coroutinen (über `qasync`) würde responsives UI-Verhalten auch bei rechenintensiven Analysen gewährleisten ohne strukturellen Umbau der Algorithmusmodule.

**Plugin-Architektur für Algorithmen.** Das aktuelle System registriert Algorithmen statisch im `algorithms`-Paket. Eine *Plugin-API* (ähnlich dem `pytest`-Plugin-System) würde es erlauben, neue Algorithmusmodule zur Laufzeit zu laden: Drittentwickler könnten eigene Betrugsmuster, Risikokennzahlen oder Optimierungsverfahren als `fylab-plugin-*`-Pakete publizieren, die automatisch in die Menüs und Widgets des Hauptfensters integriert werden.

**Parallele Subgraph-Suche.** Die  $O(n^3)$ -Hauptschleife des Subgraph Algorithmus [1] ist datenparallel: Signaturen verschiedener Knotenpaare sind unabhängig berechenbar.

Mit `numpy`-Vektorisierung (bereits teilweise realisiert) oder einer CUDA-Kernel-Implementierung (via `cupy`) ließe sich die Laufzeit auf Portfolios mit  $n > 1000$  Knoten auf Millisekunden reduzieren.

**REST-API und Microservice-Architektur.** Eine FastAPI-basierte REST-Schicht über den Algorithmusmodulen würde FyLab als Backend-Service für Web-Frontends, Mobile Apps oder institutionelle Integrationspipelines positionieren. Jedes Modul (`/api/v1/arbitrage`, `/api/v1/fraud`, `/api/v1/mst`, `/api/v1/portfolio`) exponiert seine Kernfunktionalität als zustandslosen HTTP-Endpunkt, der in Kubernetes horizontal skalierbar ist.

## 10.6 Regulatorik, Compliance und Sicherheit

**AML/KYC-Compliance-Engine.** Die Betrugsmuster-Erkennung von FyLab tangiert den regulatorischen Bereich *Anti-Money Laundering (AML)* und *Know Your Customer (KYC)*. Eine dedizierte Compliance-Erweiterung würde FATF-Rekommendationen [5]<sup>1</sup> als formale Muster in `FraudPattern`-Typen übersetzen und einen revisionssicheren Audit-Trail aller Erkennungsergebnisse führen.

**Differenzielle Privatheit für Portfolio-Analysen.** Portfolioanalysen offenbaren sensible Vermögensdaten. Durch Integration von *differenzieller Privatheit* ( $\epsilon$ -DP) in den Monte-Carlo-Simulator und die Effizienzgrenzenberechnung könnten Ergebnisse mit garantiertem Datenschutz an Dritte (z. B. Steuerberater, Banken) weitergegeben werden, ohne individuelle Positionen zu exponieren.

**Kryptographische Verifikation.** Transaktionsdaten in der eingebetteten Datenbank könnten mittels kryptographischer Hashketten (Merkel-Baum über sortierten Buchungen) gegen nachträgliche Manipulation gesichert werden – eine leichtgewichtige Auditierbarkeit abseits vollständiger Blockchain-Lösungen.

## 10.7 Wissenschaftlicher Forschungsausblick

Die in dieser Arbeit etablierten polynomiellen Reduktionen öffnen mehrere offene Forschungsfragen:

### 1. Schärfe der Subgraph-Isomorphismus-Schranke.

Gibt es finanzmathematisch relevante Teilklassen des Subgraph-Isomorphismus-Problems (z. B. planare Finanzgraphen, Baumstruktur-Portfolios, DAG-Schuldennetzwerke), für die polynomielle Algorithmen mit garantierter Korrektheit existieren? Die Theorie liefert hier: Subgraph-Isomorphismus auf Bäumen ist in  $O(n^{1.5})$  lösbar [6].

### 2. Durchschnittliche Laufzeit auf realen Finanzdaten.

Der Subgraph Algorithmus [1] hat Worst-Case  $O(n^3)$ , verhält sich empirisch jedoch nahe  $O(n^2)$  auf typischen Finanzgraphen (spärliche Adjazenzmatrizen, wenige Zyklen). Eine formale Average-Case-Analyse über Random-Graph-Modelle (Erdős-Rényi, Barabási-Albert Skalenfreiheit) wäre publikationswürdig.

---

<sup>1</sup>Die formale Komplexitätstheorie ist hier analog: AML-Regelwerke definieren verbotene Transaktionsmuster, deren Erkennung im Transaktionsgraphen ein Subgraph-ISO-Problem darstellt.

3. **Untere Komplexitätsschranken für Betrugserkennungs-Varianten.**  
Welche spezifischen Betrugsmuster-Erkennungsprobleme sind in  $P$ ? Wash-Trading auf bipartiten Graphen? Smurfing auf Sternstrukturen? Eine vollständige Klassifikation der Tracklabeled-Subgraph- Isomorphismus-Varianten nach ihrer Komplexität fehlt noch in der Literatur.
4. **Algebraische Charakterisierung der Pearson-Distanz-MSTs.**  
Mantegna [12] zeigte empirisch, dass Pearson-MSTs stabile Branching-Strukturen aufweisen. Unklar ist, welche algebraischen Eigenschaften des Korrelationstensors diese Stabilität garantieren – eine Verbindung zur Spektraltheorie positiv definiter Matrizen und zum Fiedler-Vektor des Laplacians wäre mathematisch fruchtbar.
5. **Verbindung zu Algorithmic Game Theory.**  
Schuldenausgleich via Max-Flow ist ein spezieller Fall von multi-commodity-flow. Wenn Agenten strategisch falsche Schulden deklarieren, entsteht ein unvollständiges Informations-Spiel. Die Nash-Gleichgewichtsanalyse dieses Spiels und die Frage, ob wahrheitsgemäße Deklaration eine dominante Strategie ist (Mechanism Design), ist ein offenes Problem.

Tabelle 6: Zusammenfassung der Entwicklungs- und Forschungsrichtungen

| Bereich     | Erweiterung                             | Zeithorizont  |
|-------------|-----------------------------------------|---------------|
| Algorithmen | Inkrementelles Subgraph-Matching        | kurzfristig   |
| Algorithmen | GNN-gestützte Betrugserkennung          | mittelfristig |
| Algorithmen | DRL für dynamische Asset-Allokation     | mittelfristig |
| Daten       | MT940/CSV-Import-Pipeline               | kurzfristig   |
| Daten       | Live-API-Anbindung (ECB, Alpha Vantage) | kurzfristig   |
| Daten       | Blockchain-Transaktionsgraph            | mittelfristig |
| Architektur | Qt-Threads für Async-Berechnung         | kurzfristig   |
| Architektur | REST-API / Microservice                 | mittelfristig |
| Architektur | Plugin-System für Drittmodule           | langfristig   |
| Sicherheit  | AML/KYC-Compliance-Engine               | mittelfristig |
| Sicherheit  | Differenzielle Privatheit               | langfristig   |
| Forschung   | Average-Case-Analyse Subgraph-ISO       | mittelfristig |
| Forschung   | Mechanism Design Schuldenausgleich      | langfristig   |

## Literatur

- [1] Stephan Epp. *Der Subgraph Algorithmus*, 7. Januar 2026. Verfügbar unter: <https://github.com/hjstephan86/subgraph>
- [2] J. R. Ullmann. An algorithm for subgraph isomorphism. *Journal of the ACM*, 23(1):31–42, 1976.
- [3] L. P. Cordella, P. Foggia, C. Sansone, M. Vento. A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(10):1367–1372, 2004.



- [4] S. A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC)*, S. 151–158. ACM, New York, 1972.
- [5] M. R. Garey, D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco, 1979.
- [6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 4. Auflage. MIT Press, Cambridge, MA, 2022.
- [7] R. Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.
- [8] L. R. Ford, D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956.
- [9] J. Edmonds, R. M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972.
- [10] J. B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956.
- [11] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.
- [12] R. N. Mantegna. Hierarchical structure in financial markets. *The European Physical Journal B*, 11(1):193–197, 1999.
- [13] H. Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- [14] W. F. Sharpe. Capital asset prices: A theory of market equilibrium under conditions of risk. *The Journal of Finance*, 19(3):425–442, 1964.
- [15] R. T. Rockafellar, S. Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000.
- [16] P. Artzner, F. Delbaen, J.-M. Eber, D. Heath. Coherent measures of risk. *Mathematical Finance*, 9(3):203–228, 1999.
- [17] P. Glasserman. *Monte Carlo Methods in Financial Engineering*. Springer, New York, 2004.
- [18] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.* Array programming with NumPy. *Nature*, 585:357–362, 2020.
- [19] P. Virtanen, R. Gommers, T. E. Oliphant *et al.* SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.
- [20] J. D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- [21] Riverbank Computing Limited. *PyQt6 Reference Guide*, 2023. <https://www.riverbankcomputing.com/static/Docs/PyQt6/>. Letzter Zugriff: März 2026.